

CSEN 296B-2 | Week 6 Lab Submission

AstraNotes Development Environment and First Realization Slices

Student Name: Alex Shirazi

Date: May 4, 2026

Project: AstraNotes

1. Development Direction

Language: Python 3.11

Application type: Desktop CLI (command-line interface). AstraNotes is a local-first personal note-taking application with no network dependencies. A CLI shell is the appropriate interface for the project scope: it keeps the architecture clean, makes all layers independently testable, and avoids UI framework dependencies that are not justified by any requirement.

Runtime: Python 3.11 standard library plus one third-party dependency: cryptography==42.0.5 (Apache 2.0) for Fernet symmetric encryption (SPR-01). No other external dependencies are introduced.

2. Project Structure and Build Environment

The repository structure follows the package layout defined in the Week 4.2 package diagram exactly. Each package corresponds to one architectural layer.

```
AstraNotes/  
  src/  
    astranotes/  
      models/          # Note, ValidationError, NoteNotFoundError, StoragePersistenceError  
      note.py  
      repository/      # NoteRepository interface + JsonFileRepository  
        note_repository.py  
        json_file_repository.py  
      service/         # NoteService (coordinates all operations)  
        note_service.py  
      privacy/         # PrivacyService (Fernet encryption -- placeholder this sprint)  
        privacy_service.py  
      validation/      # ValidationLayer  
        validation_layer.py  
      cli/             # CLI entry point and menu shell  
        main.py
```

```
tests/
  test_note_creation.py
planning/          # requirements, user stories, backlog, sprint-zero-plan
docs/
requirements.txt
```

To run the application:

```
cd AstraNotes/src
pip install -r ../requirements.txt
python -m astranotes.cli.main
```

To run the tests:

```
cd AstraNotes
python -m pytest tests/ -v
```

3. UI Shell

The CLI shell presents a numbered menu on each loop iteration. The shell is intentionally minimal: it surfaces the four operations covered by the first two implementation slices and defers private note management to a later sprint.

```
=====
AstraNotes | Local Notes App
=====

[1] View all notes
[2] Create a note
[3] Search notes
[4] Delete a note
[q] Quit
```

The menu loop runs until the user enters 'q'. Each option calls a dedicated handler function in `cli/main.py`, which delegates immediately to `NoteService`. The CLI layer contains no business logic: it only reads input, calls service methods, and formats output. This respects the layer separation required by NFR-02.

4. First Functionality Slices

Slice 1: Note Creation (FR-01)

`NoteService.create()` accepts a title, body, and optional tags list. It constructs a `Note` dataclass instance, passes it through `ValidationLayer.validate_note()`, and calls `NoteRepository.save()`. If the title is empty or whitespace-only, `ValidationError` is raised before any persistence occurs.

```
# astranotes/service/note_service.py
def create(self, title: str, body: str, tags: List[str] = None) -> Note:
    """FR-01: create a new plain note.
    Validates title, assigns UUID and timestamps automatically.
    Raises ValidationError if title is empty or whitespace-only."""
    note = Note(title=title, body=body, tags=tags or [])
    self._validator.validate_note(note) # raises ValidationError if invalid
    self._repo.save(note)               # raises StoragePersistenceError on disk failure
    return note
```

`ValidationLayer.validate_title()` strips whitespace before checking for an empty string, closing the edge case identified in the Week 3.1 ambiguity review (A-01): a title of `' '` is treated as empty, not as a valid three-character string.

```
# astranotes/validation/validation_layer.py
def validate_title(self, title: str) -> None:
    if not isinstance(title, str):
        raise ValidationError("title", "Title must be a string")
    if not title.strip():
        raise ValidationError("title", "Title must not be empty or whitespace-only")
```

Slice 2: Sorted Note Listing (FR-08)

`NoteService.list_sorted()` retrieves all notes from the repository and sorts them by `modified_at` descending. The stable tiebreaker (`created_at` descending) was added based on the Week 3.1 ambiguity review (A-04), which identified that equal `modified_at` values would produce non-deterministic sort order without it.

```
# astranotes/service/note_service.py
def list_sorted(self) -> List[Note]:
    """FR-08: return all notes sorted by modified_at descending.
    Stable tiebreaker: created_at descending (Week 3.1 ambiguity A-04).
    """
    notes = self._repo.list_all()
    return sorted(
        notes,
```

```

        key=lambda n: (n.modified_at, n.created_at),
        reverse=True
    )

```

Test Results

9 unit tests written against Slices 1 and 2. All tests use a temporary directory via pytest's tmp_path fixture, so no real file system state is needed (SPR-03). All 9 passed.

```

test_create_note_returns_note_with_correct_title    PASSED
test_create_note_assigns_uuid                      PASSED
test_create_note_assigns_timestamps                 PASSED
test_create_note_empty_title_raises_validation_error PASSED
test_create_note_whitespace_title_raises_validation_error PASSED
test_create_note_persists_to_disk                   PASSED
test_list_sorted_returns_most_recently_modified_first PASSED
test_list_sorted_empty_returns_empty_list           PASSED
test_list_sorted_single_note                       PASSED

```

9 passed in 0.01s

5. Traceability Note

Slice	Req IDs	UML Class / Object Evidence	UML Behavioral Evidence	Status
Note creation	FR-01, SPR-02	Note dataclass, ValidationLayer (+validate_note, +validate_title), NoteService (+create()), JsonFileRepository (+save())	Activity diagram: validation gate node, assign UUID/timestamps node, NoteRepository.save() node	Realized
Sorted listing	FR-08	Note (-modified_at, -created_at), NoteService (+list_sorted()), Object diagram: display:UI sort = modified_at DESC	UC: View Sorted List (includes Load from Disk)	Realized
Whitespace title rejection	FR-01 refined (Week 3.1 A-01)	ValidationLayer (+validate_title) -- whitespace strip before empty check	Activity diagram: validation gate (empty title raises ValidationError)	Realized
Corrupt file skip on load	FR-05 refined (Week 3.1 edge case)	JsonFileRepository.list_all() -- try/except per file, continues on JSONDecodeError	Not in activity diagram (identified as Gap 3 in Week 5.2)	Realized

Every line of implemented code traces to a named requirement ID and a named UML element. No implementation was added that could not be defended from the design artifacts.

6. Slice Review and Quality Lessons

Weaknesses found in the first-pass Copilot draft:

Readability: The first Copilot draft of `NoteService.create()` did not include docstrings or inline comments linking the method to its requirement ID. Reading it in isolation gave no indication of what FR-01 required or why `ValidationLayer` was called before `save()`. Added requirement traceability comments to every method.

Responsibility clarity: Copilot initially placed the whitespace-strip logic inside `Note.__init__()` rather than `ValidationLayer.validate_title()`. This was wrong: the `Note` model should not enforce its own validation rules. Moved the strip-and-check logic to `ValidationLayer` where it belongs per the class diagram composition relationship.

Failure handling: The first Copilot draft of `JsonFileRepository.save()` used a bare except clause that would silently swallow all exceptions. Replaced with a specific `OSError` catch that raises `StoragePersistenceError` with a message that contains no file paths or stack trace content (SPR-02).

Maintainability: Copilot generated the sort key as `lambda n: n.modified_at` with no tiebreaker. This would produce non-deterministic order for notes saved in rapid succession. Added the `created_at` tiebreaker based on the Week 3.1 ambiguity review (A-04).

Quality lesson learned:

The most valuable debugging lesson this sprint was that Copilot drafts are faster than they are correct. Every issue found in the review was a case where Copilot produced working code that violated a specific documented design decision: a responsibility boundary, a requirement constraint, or a failure handling rule from SPR-02. The design artifacts from Weeks 3 to 5 were the only reliable source of truth for catching these issues. Without them, the whitespace-strip location, the bare except, and the missing tiebreaker would all have passed a naive code review.

7. AI Reflection

What Copilot helped with:

Scaffolding the package structure. Copilot correctly suggested the `src/astranotes/` layout with separate subpackages matching the Week 4.2 package diagram. Accepted with no changes.

Generating boilerplate for the abstract `NoteRepository` interface. The ABC + abstractmethod pattern was correctly applied. Accepted.

Writing the initial JSON serialization and deserialization in `Note.to_dict()` and `Note.from_dict()`. Correct and concise. Accepted with minor additions for UTC timezone handling.

Generating the first draft of the CLI menu loop. The basic while/input/print structure was correct. Accepted as scaffolding, then refactored to extract handler functions.

What was changed:

Moved whitespace validation from `Note.__init__()` to `ValidationLayer.validate_title()` to respect layer responsibilities.

Replaced bare except with `OSError` in `JsonFileRepository` to satisfy SPR-02 failure handling requirements.

Added requirement ID comments to every method signature so the trace from code to design is explicit.

Added `created_at` tiebreaker to `list_sorted()` based on the Week 3.1 ambiguity review, which Copilot did not have context for.

Removed a Copilot suggestion to add a `NoteCache` class as a performance optimization for NFR-01. This was out of scope for Sprint Zero and would have introduced an untested, unjustified layer.